

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 1

Выполнила:

Савченко Анастасия

Группа К3341, Поток БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

На основе результатов ДЗ1 (ER-диаграмма) и ДЗ2 (OpenAPI-спецификация) реализовать работающее REST API для сервиса бронирования столиков в ресторанах. Использовать `boilerplate` на связке `Express + TypeORM + TypeScript`. Реализовать модели, соответствующие спроектированной базе данных, и контроллеры, реализующие все описанные эндпоинты.

Ход работы

1. Выбор `boilerplate` и настройка окружения

За основу взят `boilerplate express-typeorm-boilerplate`, предоставленный в материалах курса. `Boilerplate` содержит преднастроенную связку `Express + TypeORM + TypeScript` с поддержкой декораторов, JWT-аутентификации и OpenAPI-документирования.

Настроено подключение к PostgreSQL через файл `data-source.ts`. Для автоматического создания таблиц включён параметр `synchronize: true`. Параметры подключения вынесены в `.env`-файл.

Установлены зависимости: `express`, `typeorm`, `pg`, `reflect-metadata`, `jsonwebtoken`, `bcryptjs`, `dotenv`. Для разработки добавлены `typescript`, `tsx`, `ts-node` и типы.

2. Реализация моделей (Entity)

На основе ER-диаграммы из ДЗ1 созданы девять сущностей `TypeORM` в директории `src/models/`:

Entity	Таблица БД	Назначение
User	users	Пользователи системы
Restaurant	restaurants	Рестораны с адресами и ценовой категорией
Cuisine	cuisines	Справочник типов кухонь

Entity	Таблица БД	Назначение
RestaurantCuisine	restaurant_cuisines	Ассоциативная таблица M:N
Table	tables	Столики с вместимостью и зоной
MenuItem	menu_items	Блюда меню
RestaurantPhoto	restaurant_photos	Фотографии ресторанов
Booking	bookings	Бронирования столиков
Review	reviews	Отзывы с оценками

Каждая сущность использует декораторы TypeORM: `@Entity` для привязки к таблице, `@PrimaryGeneratedColumn` для автоинкрементного первичного ключа, `@Column` с указанием типа для остальных полей. Связи описаны через `@OneToMany`, `@ManyToOne` и `@JoinColumn`. Все первичные ключи именованы осмысленно: `user_id`, `restaurant_id`, `booking_id` и т.д.

Связь «многие-ко-многим» между ресторанами и кухнями реализована через ассоциативную сущность `RestaurantCuisine` с составным первичным ключом из двух внешних ключей.

3. Реализация эндпоинтов

Все эндпоинты согласно OpenAPI-спецификации из ДЗ2 реализованы в файле `src/index.ts`. Сервер поднимается на Express, прослушивает порт 3000, все маршруты имеют префикс `/api/v1`.

Auth (аутентификация):

- `POST /auth/register` — регистрация с хэшированием пароля через `bcrypt`
- `POST /auth/login` — вход с выдачей `access_token` и `refresh_token` (JWT)
- `POST /auth/refresh` — обновление токенов

Users (пользователи):

- GET /users/me — профиль текущего пользователя (защищён)
- PATCH /users/me — редактирование имени и телефона

Restaurants (рестораны):

- GET /restaurants — список с фильтрацией по городу, кухне (по ID и названию), цене и поиску по названию, с пагинацией
- GET /restaurants/:id — детальная информация с кухнями, фото и рейтингом

Cuisines (кухни):

- GET /cuisines — список всех кухонь

Dishes (блюда):

- GET /restaurants/:id/dishes — меню ресторана с фильтром по категории
- POST /restaurants/:id/dishes — добавить блюдо (защищён)
- PATCH /dishes/:id — редактировать блюдо
- DELETE /dishes/:id — удалить блюдо

Tables (столики):

- GET /restaurants/:id/tables — список столиков
- GET /restaurants/:id/tables/available — доступные столики на дату и время с учётом вместимости

Reservations (бронирования):

- GET /reservations — история бронирований пользователя
- POST /reservations — создать бронь с проверкой на конфликт по времени
- PATCH /reservations/:id/cancel — отмена брони с проверкой прав

Reviews (отзывы):

- GET /restaurants/:id/reviews — отзывы с пагинацией
- POST /restaurants/:id/reviews — создать отзыв

- PATCH /reviews/:id — редактировать (только свой)
- DELETE /reviews/:id — удалить (только свой)

Photos (фотографии):

- GET /restaurants/:id/photos — список фото
- POST /restaurants/:id/photos — добавить фото
- DELETE /photos/:id — удалить фото

4. Middleware для аутентификации

Реализован middleware auth, проверяющий JWT-токен из заголовка Authorization: Bearer При успешной проверке в объект запроса добавляется req.user с полями user_id и email. При отсутствии или невалидности токена возвращается 401.

5. Заполнение базы тестовыми данными

Создан файл seed.ts, который заполняет БД начальными данными: два пользователя, пять кухонь, три ресторана с привязками к кухням, столики и блюда. Это позволяет сразу тестировать API без ручного ввода данных.

6. Тестирование через Postman

Работоспособность всех эндпоинтов проверена через Postman. В рамках ДЗ3 создан комплексный сценарий из 11 последовательных шагов и расширенный набор из 23 тестов, покрывающих все коды ответов. Все тесты проходят успешно.

7. Структура проекта

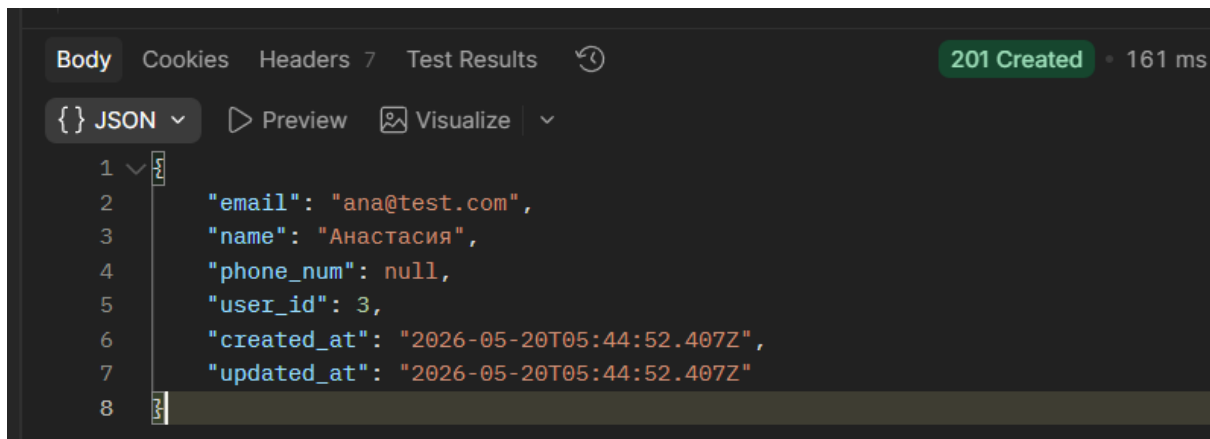
```
lab1/
├── .env                # PORT, DB_HOST, DB_USER, DB_PASS,
                        DB_NAME, JWT_SECRET
├── package.json        # зависимости и скрипты (dev, build, start)
├── tsconfig.json       # TypeScript с emitDecoratorMetadata и
                        experimentalDecorators
```

```
└─ src/
    ├── index.ts      # точка входа: Express-сервер и все эндпоинты
    ├── data-source.ts # конфигурация TypeORM
    ├── seed.ts       # заполнение БД тестовыми данными
    ├── models/       # 9 entity-файлов TypeORM
    ├── controllers/  # логика контроллеров
    ├── routes/       # маршрутизация
    └─ middlewares/   # JWT-auth middleware
```

Вывод

В результате выполнения лабораторной работы реализован полностью работающий REST API для сервиса бронирования столиков в ресторанах. На основе ER-диаграммы из ДЗ1 созданы девять моделей TypeORM, отражающих все сущности и связи. На основе OpenAPI-спецификации из ДЗ2 реализованы все заявленные эндпоинты с поддержкой фильтрации, пагинации, JWT-аутентификации и обработки ошибок. API протестировано через Postman в рамках ДЗ3. Проект готов к расширению и развёртыванию.

<http://localhost:3000/api/v1/auth/register>



<http://localhost:3000/api/v1/auth/login>

